Technical Documentation on RAG based customer support assistant

ABSTRACT:

The RAG Based Customer Support Assistant is an AI-powered system developed to automate customer support queries using Retrieval-Augmented Generation (RAG). The project is designed to answer common user questions related to login issues, order tracking, refunds, Prime membership, and payment problems.

The system processes PDF-based support manuals, converts the content into embeddings, and stores them in a ChromaDB vector database. When a customer enters a query, the system retrieves relevant document chunks and uses the Groq-hosted llama-3.3-70b-versatile model to generate an accurate response.

LangGraph is used to control workflow routing, while Human-in-the-Loop (HITL) escalation is included for complex or sensitive issues. This project demonstrates practical use of LLMs, vector search, workflow automation, and responsible AI in customer support.

Introduction:

What is RAG

Retrieval-Augmented Generation (RAG) is an AI approach that combines document retrieval with a Large Language Model. It first searches relevant information from stored documents, then generates answers using that context. This improves accuracy, relevance, and reduces incorrect responses compared to standalone AI models.

Why it is Needed

Traditional chatbots rely on fixed rules, while normal AI models may produce inaccurate answers. RAG is needed to provide trustworthy responses using updated company documents. It helps reduce hallucinations, improves customer satisfaction, and ensures responses follow official support policies.

Use Case Overview

This project uses RAG to build a Customer Support Assistant for Amazon services. It answers queries related to login, Prime membership, orders, refunds, and payments. Complex or sensitive issues are escalated to human support for better resolution.

Problem Statement

Customer support teams often receive a large number of repetitive queries related to login issues, order tracking, refunds, Prime membership, and payment problems. Handling these requests manually increases response time, workload, and operational cost. Traditional chatbots may fail to understand natural language queries or provide inaccurate answers. Therefore, there is a need for an intelligent support system that can retrieve correct information from company documents, generate accurate responses, and escalate complex issues to human agents when necessary.

Technology Stack

- Python
- Streamlit
- LangChain
- LangGraph
- ChromaDB
- HuggingFace Embeddings
- Groq API
- llama-3.3-70b-versatile
- PyPDFLoader
- Sentence Transformers

System Architecture Explanation:

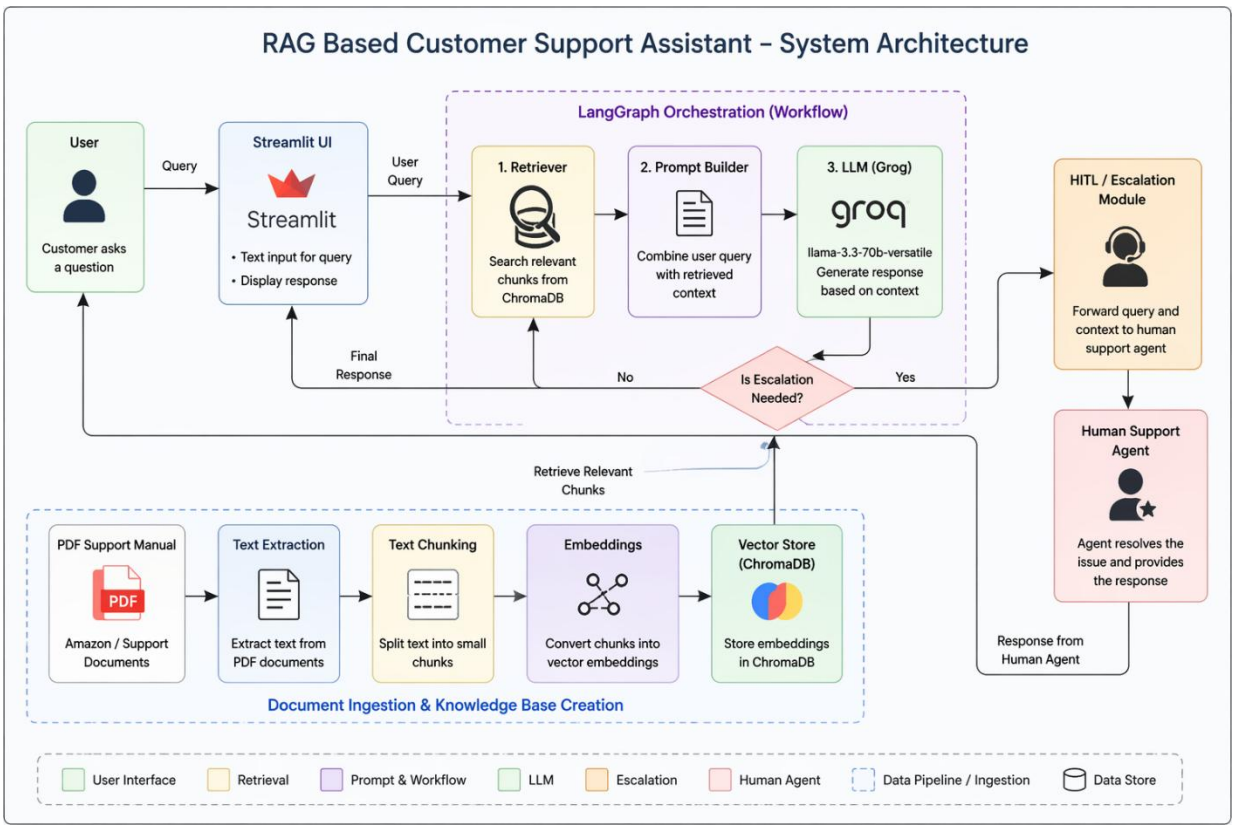
HLD

The High-Level Design (HLD) of the **RAG Based Customer Support Assistant** represents the overall architecture of the system. It includes a **Streamlit web interface** for user interaction, a **PDF document ingestion pipeline** for loading the support manual, an

embedding model for converting text into vectors, and **ChromaDB** for storing embeddings. It also includes a **retrieval layer**, the **llama-3.3-70b-versatile** language model for answer generation, **LangGraph** for workflow orchestration, and a **Human-in-the-Loop (HITL)** module for escalations.

Component Interactions

All components work in sequence to process user queries. First, the user enters a question through the Streamlit interface. The retriever searches ChromaDB and fetches the most relevant document chunks. These chunks are combined with the user query and sent to the LLM for generating an accurate response. LangGraph controls the flow between retrieval, response generation, and escalation. If the query is complex, low-confidence, or sensitive, the HITL module forwards it to a human support agent.



Design Decisions

Chunk Size Choice

Chunk size is an important factor in RAG systems because large documents cannot be sent entirely to the language model. The support manual is divided into smaller text chunks to improve search efficiency. A moderate chunk size with overlap is used so that important context is preserved while maintaining accurate retrieval.

Embedding Strategy

The project uses a sentence-transformer embedding model to convert text chunks into numerical vectors. These embeddings capture semantic meaning instead of exact keyword matching. This helps the system understand similar queries such as “money deducted” and “payment failed” even when the wording is different.

Retrieval Approach

The retrieval layer uses semantic similarity search through ChromaDB. When a user enters a question, the query is converted into an embedding and compared with stored document vectors. The top relevant chunks are selected and passed to the language model as context for response generation.

Prompt Design Logic

The prompt is designed to instruct the model to answer only from the retrieved context. It also asks the model to provide clear, concise, and professional responses. This reduces hallucinations and ensures answers remain aligned with the customer support manual.

Workflow Explanation

LangGraph Usage

LangGraph is used to manage the execution flow of the RAG Based Customer Support Assistant. It helps organize the system into connected nodes where each node performs a specific task. This makes the workflow modular, easier to maintain, and suitable for handling routing decisions such as normal query processing or escalation.

Node Responsibilities

Each node has a defined responsibility in the workflow. The Input Node receives the customer query. The Retrieval Node searches ChromaDB and fetches relevant document chunks. The Generation Node sends the query and retrieved context to the LLM for answer creation. The Output Node displays the final response. The Escalation Node handles Human-in-the-Loop support requests for critical issues.

State Transitions

State transitions represent the flow of data between nodes during execution. The state usually contains fields such as user question, retrieved context, generated answer, and confidence score. Each node reads the current state, updates the required values, and passes the modified state to the next node until the workflow is completed.

Conditional Logic

Intent Detection

Intent detection is used to identify the purpose of the customer query. The system analyzes the user input to determine whether the request is related to login issues, order tracking, refund status, Prime membership, payment problems, or complaints. It also detects sensitive intents such as fraud, unauthorized transactions, or requests for human support. This helps the assistant understand how the query should be handled.

Routing Decisions

Routing decisions are made based on the detected intent and confidence level. If the query is simple and relevant document chunks are available, it is sent through the normal retrieval and response generation path. If the query is complex, low-confidence, or sensitive, the system routes it to the Human-in-the-Loop (HITL) module for manual support. This ensures both efficiency and reliability.

HITL Implementation

Role of Human Intervention

Human-in-the-Loop (HITL) is included to handle cases where automated responses may not be sufficient. When the system detects sensitive, complex, or low-confidence queries such as fraud complaints, payment disputes, repeated unresolved issues, or requests for a manager, the query is escalated to a human support agent. The agent reviews the issue, verifies details, and provides the final resolution. This ensures critical customer concerns are handled with proper judgment and accountability.

Benefits and Limitations

The main benefit of HITL is improved trust, accuracy, and customer satisfaction for complex issues. It reduces the risk of incorrect automated responses and supports better decision-making. However, HITL also has limitations. Human intervention

increases response time compared to instant AI replies and requires trained support staff. Managing escalations efficiently is necessary to maintain service quality.

Challenges & Trade-offs

Retrieval Accuracy vs Speed

Increasing the number of retrieved chunks can improve answer accuracy because the model receives more relevant context. However, retrieving more data also increases processing time and response latency. A balance is required so the system remains both accurate and fast enough for customer support use.

Chunk Size vs Context Quality

Small chunks improve retrieval precision because the search focuses on specific information. However, they may lose important surrounding context. Large chunks preserve more context and complete meaning, but they may include irrelevant content. Choosing an optimal chunk size is important for effective performance.

Cost vs Performance

Using larger language models can provide better reasoning and higher quality responses, but API usage cost becomes higher. Smaller models are more affordable and faster, but may reduce response quality. The system must balance budget, speed, and answer quality based on business requirements.

Testing Strategy

Testing Approach

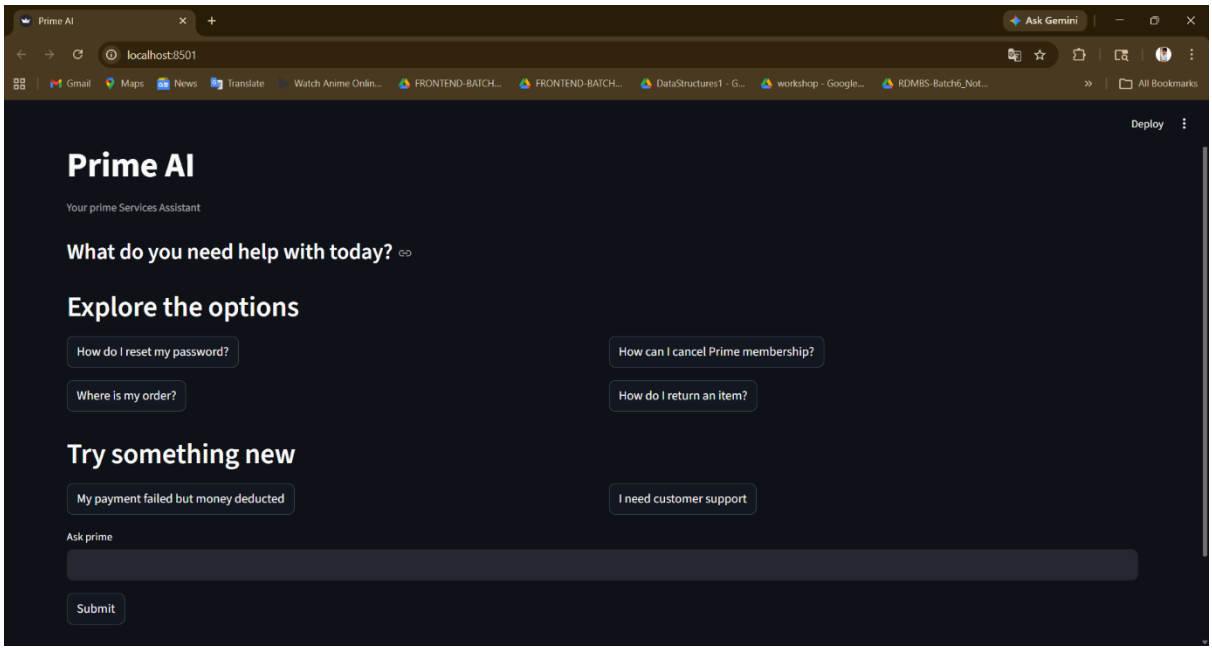
The system is tested using functional, performance, and routing-based testing methods. Functional testing checks whether the assistant gives correct answers for common customer support queries. Performance testing measures response speed and retrieval efficiency. Routing testing verifies whether normal queries are answered automatically and sensitive or complex queries are escalated to the Human-in-the-Loop (HITL) module. Error testing is also performed for blank inputs, missing context, or failed API responses.

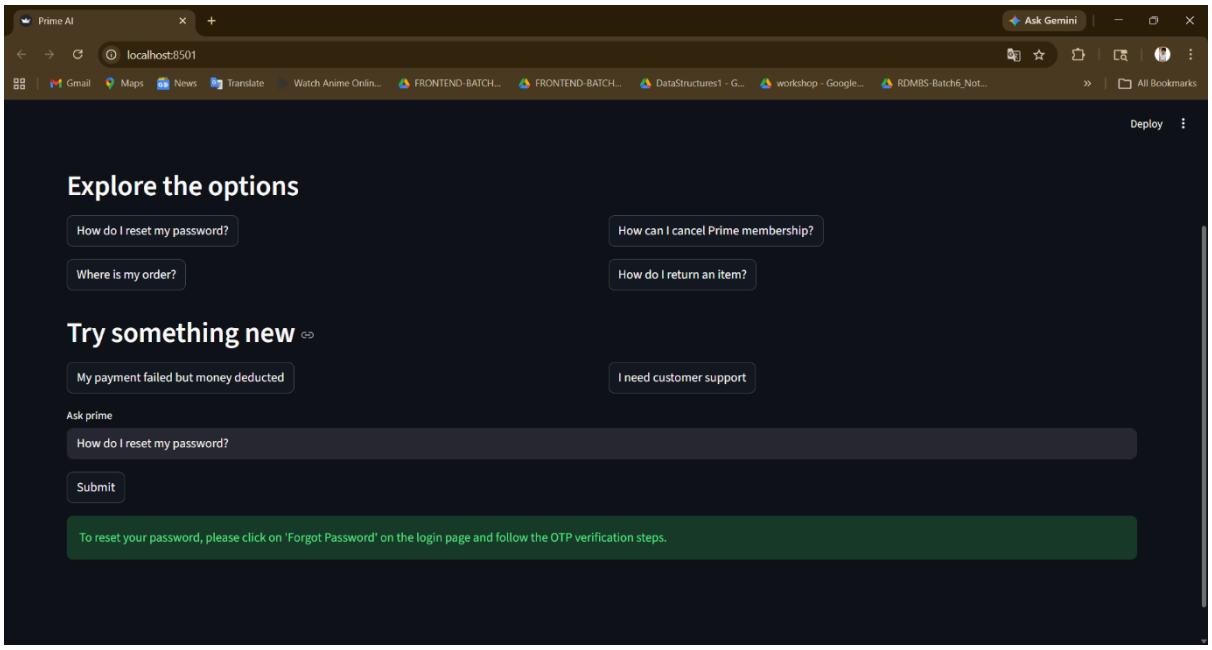
Sample Queries

- How do I reset my password?
- Where is my order?
- How can I cancel Prime membership?
- My payment failed but money was deducted.

- How do I request a refund?
- I need to speak with customer support.
- My account is locked after multiple login attempts.

Screenshots





Future Enhancements

Multi-document Support

The system can be enhanced to support multiple documents such as billing manuals, return policies, technical guides, and department-specific knowledge bases. This will allow the assistant to answer a wider range of customer queries with better accuracy.

Feedback Loop

A feedback mechanism can be added where users rate responses as helpful or unhelpful. This feedback can be used to improve prompts, retrieval quality, and overall system performance over time.

Memory Integration

Conversation memory can be integrated so the assistant remembers previous user interactions within a session. This enables more natural follow-up conversations and personalized support experiences.

Deployment

The project can be deployed on cloud platforms such as Render, AWS, or Azure for real-time public access. Deployment would allow multiple users to interact with the assistant through a live web application.